

PRACTICAL WEEK-5

1. Implement a producer-consumer communication system using anonymous pipes where the parent process generates data and the child process consumes it. Measure the communication efficiency.

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>
#include <time.h>

#define SIZE 1000

int main()
{
    int pipefd[2];
    pid_t pid;
    int data[SIZE];

    if (pipe(pipefd) == -1)
    {
        perror("pipe");
        return 1;
    }

    pid = fork();

    if (pid < 0)
    {
        perror("fork");
        return 1;
    }

    if (pid > 0)
    {
        // Parent - Producer
        close(pipefd[0]);

        for (int i = 0; i < SIZE; i++)
        {
            data[i] = i + 1;
        }
    }
}
```

2.

```

if (pid > 0)
{
    // Parent - Producer
    close(pipefd[0]);

    for (int i = 0; i < SIZE; i++)
    {
        data[i] = i + 1;
    }

    clock_t start = clock();

    write(pipefd[1], data, sizeof(data));

    clock_t end = clock();

    close(pipefd[1]);

    wait(NULL);

    double time_taken =
        (double)(end - start) / CLOCKS_PER_SEC;

    printf("\nProducer: Generated %d integers.\n", SIZE);
    printf("Producer: Data sent through pipe.\n");
    printf("Communication time: %f seconds\n", time_taken);

    if (time_taken > 0)
    {
        double efficiency =
            SIZE / time_taken;

        printf("Communication efficiency: %f integers/second\n",
            efficiency);
    }
}
else

```

3.

```

    }
    else
    {
        // Child - Consumer
        close(pipefd[1]);

        int received[SIZE];

        read(pipefd[0], received, sizeof(received));

        close(pipefd[0]);

        printf("Consumer: Data received successfully.\n");
        printf("First 10 values received:\n");

        for (int i = 0; i < 10; i++)
        {
            printf("%d ", received[i]);
        }

        printf("\n");
    }

    return 0;
}

```

^G Help	^O Write Out	^W Where Is	^K Cut	^T Execute
^X Exit	^R Read File	^_ Replace	^U Paste	^J Justify

4.

```

laasya@laasya:~$ cd seesion5
laasya@laasya:~/seesion5$ gedit producer_consumer.c
Command 'gedit' not found, but can be installed with:
sudo snap install gedit # version 48.1, or
sudo apt install gedit # version 46.1-3
See 'snap info gedit' for additional versions.
laasya@laasya:~/seesion5$ nano producer_consumer.c
laasya@laasya:~/seesion5$ gcc producer_consumer.c -o producer_consumer
laasya@laasya:~/seesion5$ ./producer_consumer
Consumer: Data received successfully.
First 10 values received:
1 2 3 4 5 6 7 8 9 10

Producer: Generated 1000 integers.
Producer: Data sent through pipe.
Communication time: 0.000008 seconds
Communication efficiency: 125000000.000000 integers/second
laasya@laasya:~/seesion5$ |

```

2. Develop a program that executes the equivalent of the shell command: `ls -l | grep "."` using `fork()`, `pipe()`, `dup2()`, and `exec()` system calls.

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>

int main()
{
    int pipefd[2];
    pid_t pid1, pid2;

    // Create pipe
    if (pipe(pipefd) == -1)
    {
        perror("pipe");
        return 1;
    }

    // First child process
    pid1 = fork();

    if (pid1 < 0)
    {
        perror("fork");
        return 1;
    }

    if (pid1 == 0)
    {
        // Child 1: ls -l

        close(pipefd[0]);

        dup2(pipefd[1], STDOUT_FILENO);

        close(pipefd[1]);

        execlp("ls", "ls", "-l", (char *)NULL);
    }
}
```

```

        perror("exec ls");
        exit(1);
    }

    // Second child process
    pid2 = fork();

    if (pid2 < 0)
    {
        perror("fork");
        return 1;
    }

    if (pid2 == 0)
    {
        // Child 2: grep "."

        close(pipefd[1]);

        dup2(pipefd[0], STDIN_FILENO);

        close(pipefd[0]);

        execlp("grep", "grep", ".", (char *)NULL);

        perror("exec grep");
        exit(1);
    }

    // Parent process
    close(pipefd[0]);
    close(pipefd[1]);

    waitpid(pid1, NULL, 0);
    waitpid(pid2, NULL, 0);

```

```

laasya@laasya:~/seesion5$ nano ls_grep.c
laasya@laasya:~/seesion5$ gcc ls_grep.c -o ls_grep
laasya@laasya:~/seesion5$ ./ls_grep
total 44
-rwxr-xr-x 1 laasya laasya 16352 Sep 16 06:51 ls_grep
-rw-r--r-- 1 laasya laasya 1171 Sep 16 06:50 ls_grep.c
-rwxr-xr-x 1 laasya laasya 16440 Sep 16 06:49 producer_consumer
-rw-r--r-- 1 laasya laasya 1606 Sep 16 06:49 producer_consumer.c

Pipeline execution completed.
laasya@laasya:~/seesion5$ |

```